

สรุปเทคนิคทั้งหมด

ขั้นตอน	เทคนิค	Library	ผลลัพธ์ของเทคนิค	เหตุผลที่ใช้
Image Preprocessing	Gaussian Blur	OpenCV	ภาพมี noise ลดลงและภาพเรียบขึ้น	ลดสัญญาณรบกวนก่อนการตรวจจับขอบ
Image Preprocessing	Canny Edge Detection	OpenCV	ได้ภาพขอบของวัตถุในภาพ	ใช้ตรวจจับขอบของป้ายทะเบียน
Image Preprocessing	Contour Detection	OpenCV	ตรวจพบรูปร่างของป้ายทะเบียน	ใช้หา bounding shape ของป้ายทะเบียน
Image Preprocessing	Perspective Transform	OpenCV	ภาพป้ายถูกปรับให้ตรงเป็นสี่เหลี่ยม	แก้ปัญหาป้ายเอียงจากมุมกล้อง
Image Preprocessing	PCA Alignment (minAreaRect)	OpenCV, NumPy	ปรับแนวของภาพให้ตัวอักษรอยู่ในแนวนอน	ลดความเอียงของตัวอักษรเพื่อช่วย OCR
Image Preprocessing	Image Upscaling (Resize)	OpenCV	ภาพมีขนาดใหญ่ขึ้นและรายละเอียดชัดขึ้น	ช่วยให้ OCR อ่านตัวอักษรเล็กได้ดีขึ้น
Image Preprocessing	Bilateral Filtering	OpenCV	ลด noise แต่ยังคงขอบของตัวอักษร	ป้องกันตัวอักษรถูก blur
Image Preprocessing	CLAHE (Contrast Enhancement)	OpenCV	เพิ่ม contrast ของตัวอักษรในภาพ	ทำให้ตัวอักษรเด่นจากพื้นหลัง
Image Preprocessing	Image Sharpening	OpenCV	ขอบของตัวอักษรคมชัดขึ้น	เพิ่มความแม่นยำของ OCR
OCR Detection	YOLO Object Detection	Ultralytics YOLO	ตรวจจับตำแหน่งของตัวอักษรและตัวเลข	ใช้ deep learning ในการ detect character
OCR Post-processing	Bounding Box Decoding	Ultralytics, NumPy	ได้ตำแหน่ง (x,y) และ class ของตัวอักษร	ใช้เตรียมข้อมูลสำหรับ parsing
OCR Post-processing	Row Clustering (K-Means)	Scikit-learn	แยกตัวอักษรเป็น 2 บรรทัด (เลขทะเบียน / จังหวัด)	ป้ายทะเบียนไทยมี 2 บรรทัด
Parsing	Spatial Sorting	Python	ตัวอักษรถูกเรียงตามตำแหน่งซ้าย→ขวา	ใช้สร้างลำดับตัวอักษรที่ถูกต้อง
Parsing	Duplicate Removal	Python	ลบ bounding box ที่ซ้ำ	ลดการอ่านตัวอักษรซ้ำจาก OCR
Parsing	Character Mapping	Python Dictionary	แปลง class label เป็นตัวอักษรจริง	แปลง label ของโมเดลเป็นภาษาไทย
Parsing	Plate Format Normalization	Python	ได้รูปแบบป้ายทะเบียนที่ถูกต้อง เช่น 1กข234	ใช้กฎของป้ายทะเบียนไทยเพื่อแก้ OCR error

1. เทคนิคใน Pre-processing Pipeline

ขั้นตอนหลักประกอบด้วย

1.1 Grayscale Conversion

เทคนิค : แปลงภาพสีเป็นภาพขาวดำ

Library : OpenCV (cv2)

โค้ดที่ใช้ : `gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`

เหตุผล

- ลดจำนวน channel ของภาพ
- ทำให้การประมวลผลเร็วขึ้น
- ทำให้ edge detection ทำงานได้ดีขึ้น

1.2 Gaussian Blur

เทคนิค : การทำ smoothing เพื่อลด noise

Library : OpenCV

โค้ด : `blur = cv2.GaussianBlur(gray, (5,5), 0)`

เหตุผล : ลด noise ก่อนทำ edge detection เพื่อป้องกันการตรวจจับขอบผิด

1.3 Canny Edge Detection

เทคนิค : ตรวจจับขอบของวัตถุในภาพ

Library : OpenCV

โค้ด : `edged = cv2.Canny(blur, 50, 200)`

เหตุผล

ใช้ค้นหา contour ของป้ายทะเบียน

1.4 Contour Detection

เทคนิค : ตรวจจับรูปร่างของวัตถุในภาพ

Library : OpenCV

โค้ด : `contours, _ = cv2.findContours(edged, cv2.RETR_LIST, cv2.CHAIN_APPROX_SIMPLE)`

เหตุผล : ใช้ค้นหากรอบของป้ายทะเบียนเพื่อทำ perspective transform

1.5 Perspective Transform (Deskew)

เทคนิค : ปรับมุมของภาพให้ตรง

Library : OpenCV

โค้ด : `cv2.getPerspectiveTransform()`

`cv2.warpPerspective()`

เหตุผล : ป้ายทะเบียนมักถูกถ่ายในมุมเอียง การปรับ perspective ทำให้ OCR อ่านได้ง่ายขึ้น

1.6 PCA-based Rotation Alignment

เทคนิค : การปรับ orientation ของภาพด้วย Principal Component Analysis

Library : OpenCV, NumPy

โค้ด

- `cv2.minAreaRect()`
- `cv2.getRotationMatrix2D()`
- `cv2.warpAffine()`

เหตุผล : หมุนภาพให้ตัวอักษรอยู่ในแนวนอน

1.7 Image Upscaling

เทคนิค : ขยายขนาดภาพ

Library : OpenCV

โค้ด : `cv2.resize(image, fx=scale, fy=scale, interpolation=cv2.INTER_CUBIC)`

เหตุผล : ตัวอักษรขนาดเล็กทำให้ OCR อ่านยาก การ upscale ทำให้รายละเอียดชัดขึ้น

1.8 Bilateral Filtering

เทคนิค : ลด noise แต่รักษารูปร่างของภาพ

Library : OpenCV

โค้ด : `cv2.bilateralFilter(resized, d=7, sigmaColor=55, sigmaSpace=55)`

เหตุผล : ช่วยให้ตัวอักษรยังคงชัดหลังการลด noise

1.9 CLAHE (Contrast Enhancement)

เทคนิค : Contrast Limited Adaptive Histogram Equalization

Library : OpenCV

โค้ด : `cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))`

เหตุผล : เพิ่ม contrast ของตัวอักษรในพื้นที่เล็ก ๆ

1.10 Image Sharpening

เทคนิค : เพิ่มความคมของตัวอักษร

Library : OpenCV

โค้ด : `cv2.addWeighted()`

เหตุผล : ทำให้เส้นของตัวอักษรชัดเจนสำหรับ OCR

2. เทคนิคใน OCR Post-processing

หลังจาก YOLO OCR ตรวจจับตัวอักษร ระบบต้องจัดการข้อมูล bounding box

2.1 Bounding Box Decoding

เทคนิค : แปลงผลลัพธ์จาก YOLO model เป็นโครงสร้างข้อมูล

Library

- Ultralytics YOLO
- NumPy

โค้ด

```
ocr_r.bboxes.xyxy
```

```
ocr_r.bboxes.cls
```

```
ocr_r.bboxes.conf
```

2.2 Row Clustering

เทคนิค : ใช้ K-Means clustering เพื่อแยกบรรทัดของป้ายทะเบียน

Library : scikit-learn

โค้ด : KMeans(n_clusters=2)

เหตุผล

ป้ายทะเบียนไทยมี

บรรทัดบน → หมายเลขทะเบียน

บรรทัดล่าง → จังหวัด

3. เทคนิคใน Parsing

Parsing เป็นขั้นตอนสำคัญในการ จัดเรียงตัวอักษร OCR ให้เป็นเลขทะเบียนที่ถูกต้อง

3.1 Sorting by Spatial Position

เทคนิค : เรียงตัวอักษรตามตำแหน่งในภาพ

Library

- Python
- NumPy

โค้ด : `sorted(items, key=lambda x: x["x_center"])`

เหตุผล : ตัวอักษรถูกอ่านจากซ้ายไปขวา

3.2 Duplicate Removal

เทคนิค : ลบ bounding box ที่ซ้ำกัน

Library : Python

โค้ด : `clean_items()`

วิธี เปรียบเทียบตำแหน่ง

$$|\Delta x| < 8$$

$$|\Delta y| < 8$$

3.3 Character Mapping

เทคนิค : แปลง class label เป็นตัวอักษร

Library : Python dictionary

ตัวอย่าง

A01 → ก

A02 → ข

BKK → กรุงเทพมหานคร

3.4 Plate Format Normalization

เทคนิค : ใช้กฎของป้ายทะเบียนไทย

กฎ

prefix digit ≤ 1

letters ≤ 2

numbers ≤ 4

ตัวอย่าง

OCR raw → [1 ก ข 2 3 4]

Result → 1กข234